



What is DVC?

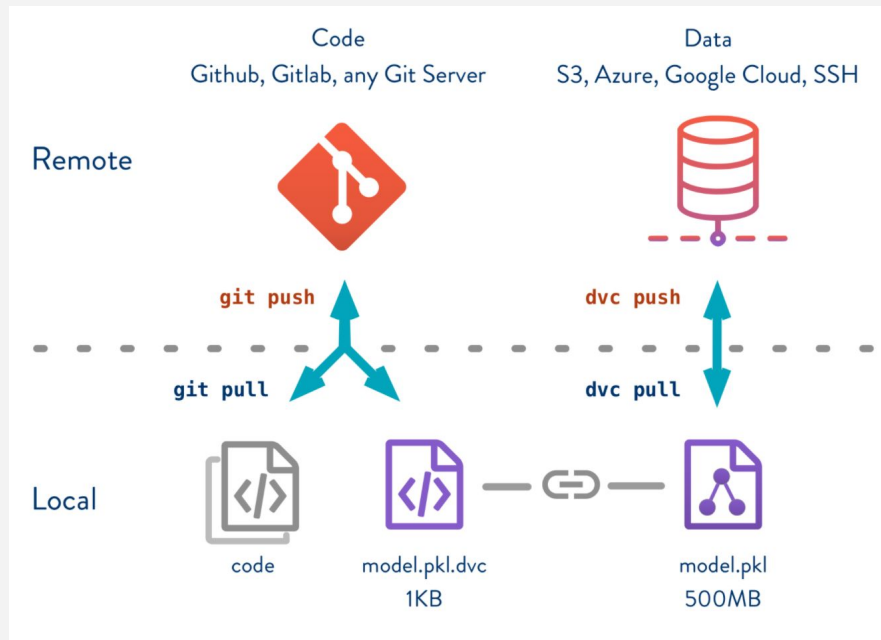
What is DVC?^[ref]

- **Simple command line Git-like experience.**
 - Does not require installing and maintaining any databases.
 - Does not depend on any proprietary online services.
- Management and **versioning** of **datasets and ML models**.
 - Data is saved in S3, Google cloud, Azure, SSH server, HDFS, or even local HDD RAID.
- Makes **projects reproducible** and shareable; answers questions on **how a model was built**.
- Helps **manage experiments with Git tags/branches** and **metrics tracking**.

*“DVC aims to replace spreadsheet and document sharing tools (such as Excel or Google Docs) which are being used frequently as both **knowledge repositories and team ledgers**.*

DVC also replaces both ad-hoc scripts to track, move, and deploy different model versions; as well as ad-hoc data file suffixes and prefixes.”

What is DVC?



- **dvc and git**

- **git:** version code, small files
- **dvc:** version data, intermed. results, models
- dvc uses **git**, w/o storing file content in repo

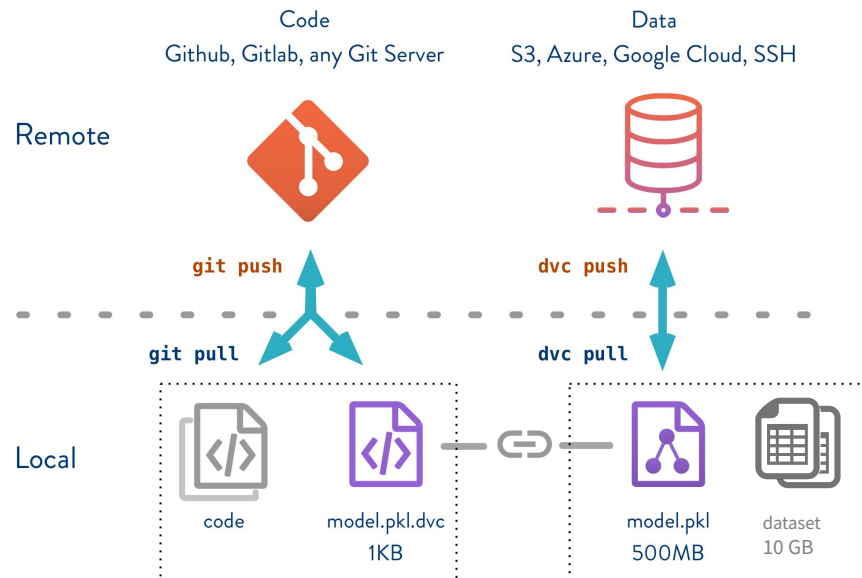
- **versioning and storing large files**

- dvc save info on data in special **.dvc** files
- **.dvc** files can then be versioned using **git**
- actual storage happens w remote storage
- dvc supports many remote storage types

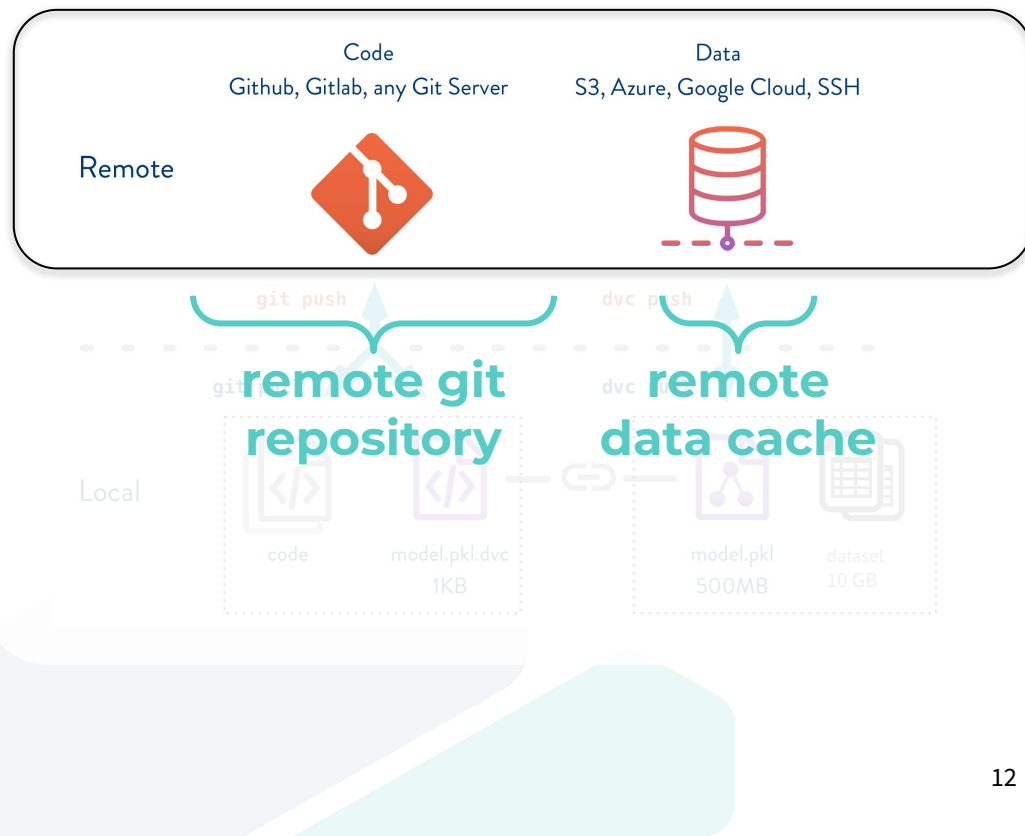
- **dvc main features**

- data versioning
- data access
- data pipelines

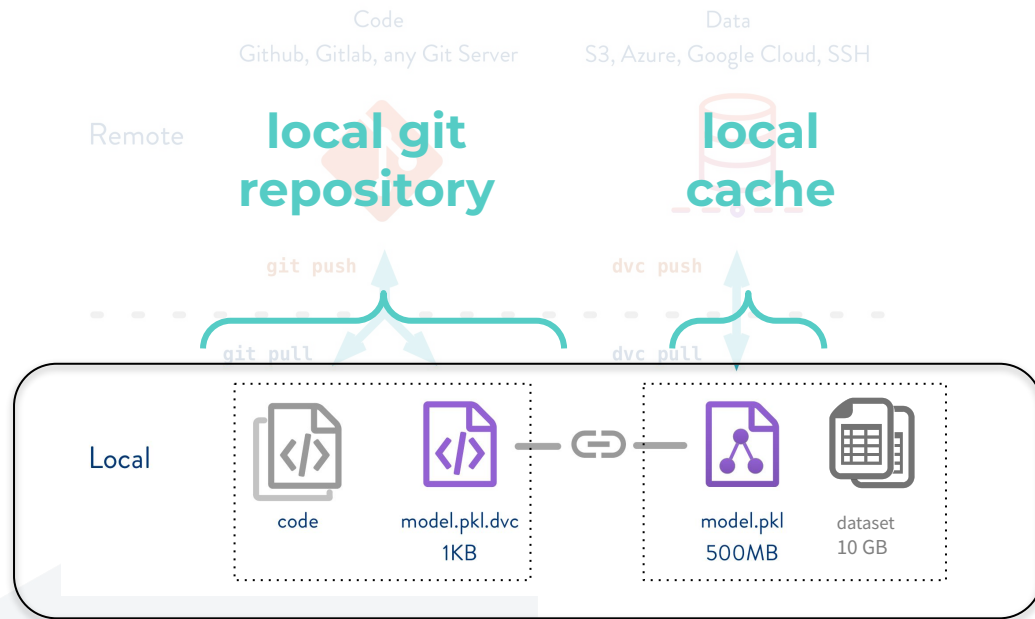
How DVC works with data?



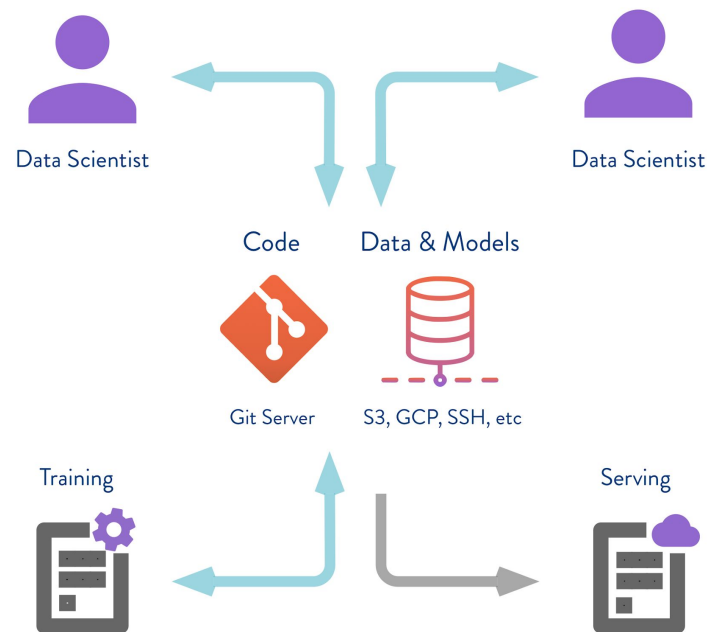
Store data in remote storage



Bring data to local workspace



Simplify a team collaboration





Getting Started

Install

- Install as a python package.

```
$ pip install dvc
```

- Depending on remote storage you will use, you may want to install specific dependencies.

```
$ pip install dvc[s3] # support Amazon S3  
$ pip install dvc[ssh] # support ssh  
$ pip install dvc[all] # all supports
```

Initialization

- We must work inside a **Git repository**. If it does not exist yet, we create and initialize one.

```
$ mkdir ml_project & cd ml_project
$ git init
```

- **Initializing a DVC project** creates and automatically `git add` a few important files.

```
$ dvc init
$ git status -s
A .dvc/.gitignore
A .dvc/config
A .dvc/plots/confusion.json
A .dvc/plots/default.json
A .dvc/plots/scatter.json
A .dvc/plots/smooth.json
A .dvcignore

$ git commit -m "Initialize dvc project"
```

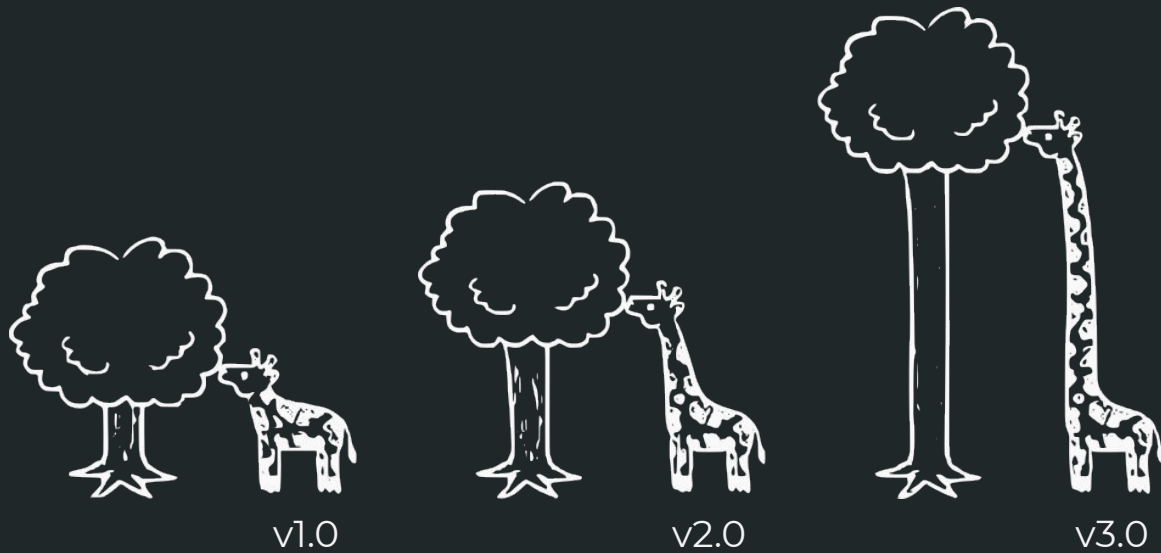
Tell Git not to track `.dvc/cache` and `.dvc/tmp`

TOML file with configurations for

- dvc remote storage — name, url
- dvc cache – reflink/copy/hardlink, location, ...
- ...

Arbitrary files that you may create yourselves.
~~Plot templates (visualize & compare metrics)~~

Tell dvc what not to track (empty for now)



Data Versioning

Getting some data

- Let's download some data to train and validate a "cat VS dog" CNN classifier.
We use `dvc get`, which is like `wget` to download data/models from a remote dvc repo.

Use "tutorials/versioning/data.zip" instead

```
$ dvc get https://github.com/iterative/dataset-registry tutorial/ver/data.zip
$ unzip data.zip & rm -f data.zip
inflating: data/train/cats/cat.001.jpg
...
```

- This folder contains 43 MB of JPG figures organized in a hierarchical fashion.

```
data
├── train
│   ├── dogs      # 500 pictures
│   └── cats      # 500 pictures
└── validation
    ├── dogs      # 400 pictures
    └── cats      # 400 pictures
```

`tree -d data`

Start versioning data

- Tracking data with DVC is very similar to tracking code with git.

```
$ dvc add data/
100% Add|██████████|1/1 [00:30, 30.51s/file]
To track the changes with git, run:
    git add .gitignore data.dvc

$ git add .gitignore data.dvc
$ git commit -m "Add first version of data/"
$ git tag -a "v1.0" -m "data v1.0, 1000 images"
```

Tell git not to track the `data/` directory

DVC-generated, contains hash to track `data/` :

```
outs:
- md5: b8f4d5a78e55e88906d5f4aeaf43802e.dir
  path: data
```

→ human readable, can be versioned with git!

- Quite a few things happened when calling `dvc add` :
 - The **hash** of the content of `data/` was computed and added to a new `data.dvc` file
 - DVC updates `.gitignore` to tell Git **not to track the content of `data/`**
 - The **physical content** of `data/` —i.e. the jpg images— **has been moved to a cache** (by default the cache is located in `.dvc/cache/` but using a remote cache is possible!)
 - The files were **linked back to the workspace** so that it looks like nothing happened (the user can configure the link type to use: hard link, soft link, reflink, copy)

Make changes to tracked data (add)

- Let's download some more data for our “cat VS dog” dataset.
Running `dvc diff` will confirm that dvc is aware the data has changed!

~~"tutorials/versioning/new-labels.zip"~~

```
$ dvc get https://github.com/iterative/dataset-registry tutorial/ver/new-labels.zip
$ unzip new-labels.zip & rm -f new-labels.zip
  inflating: data/train/cats/cat.501.jpg
...
$ dvc diff
Modified:
  data/
```

- To track the changes in our data with dvc, we follow the same procedure as before.

```
$ dvc add data/
$ git diff data.dvc
outs:
-- md5: b8f4d5a78e55e88906d5f4aeaf43802e.dir
+- md5: 21060888834f7220846d1c6f6c04e649.dir
  path: data
$ git commit -am "New version of data/ with more training images"
$ git tag -a "v2.0" -m "data v2.0, 2000 images"
```

Switch between versions



- To switch version, first run **git checkout**. This **affects data.dvc but not workspace files in data/ !**

```
$ git checkout v1.0
$ dvc diff
Modified:
  data/
```

- To fix this mismatch we simply call **dvc checkout**. This reads the cache and **updates the data in the workspace based on the current *.dvc files**.

```
$ dvc checkout
M      data/
$ dvc status
Data and pipelines are up to date.
```



Working with Storages

Configure remote storage

- A **remote storage** is for dvc, what a GitHub is for git:
 - push and pull files from your workspace to the remote
 - easy sharing between developers
 - safe backup should you ever do a terrible mistake à la `rm -rf *`
- Many remote storages are supported (Google Drive, Amazon S3, Google Cloud, SSH, HDFS, HTTP, ...) But we (as for Git) nothing prevents us to use a “local remote”!

```
$ mkdir -p ~/tmp/dvc_storage
$ dvc remote add --default loc_remote ~/tmp/dvc_storage
Setting 'loc_remote' as a default remote.
```

```
$ git add .dvc/config
$ git commit -m "Configure remote storage loc_remote"
```

DVC-generated, contains remote storage config

```
[core]
  remote = loc_remote
['remote "loc_remote"']
  url = /root/tmp/dvc_storage
```

<https://doc.dvc.org/user-guide/data-management/remote-storage/ssh>

Storing, sharing, retrieving from storage

- Running basically `dvc push` uploads the content of the cache to the remote storage. This is pretty much like `git push`.

```
$ dvc push
1800 files pushed
```

- Now, even if all the data is deleted from our workspace and cache, we can download it with `dvc pull`. This is pretty much like `git pull`.

```
$ rm -rf .dvc/cache data
$ dvc pull # update .dvc/cache with contents from remote
1800 files fetched

$ dvc checkout # update workspace, linking data from .dvc/cache
A data/
```

Access data from storage

- First, we can explore the content of a DVC repo hosted on a Git server.

```
$ dvc list https://github.com/iterative/dataset-registry
README.md
get-started/
tutorial/
...
```

- When working outside of a DVC project —e.g. in automated ML model deployment— use **dvc get**

```
$ dvc get https://github.com/iterative/dataset-registry tutorial/ver/new-labels.zip
```

- When working inside of another DVC project, we want to keep the connection between the projects. In this way, others can know where the data comes from and whether new versions are available.

```
$ dvc import https://github.com/iterative/dataset-registry tutorial/ver/new-labels.zip
$ git add new-labels.zip.dvc .gitignore
$ git commit -m "Import data from source"
```

dvc import is like `dvc get + dvc add`, but the resulting `data.dvc` also includes a ref to the source repo!

- Note.** For all these commands we can specify a git revision (sha, branch, or tag) with `--rev <commit>`.

Data Registries

- We can build a DVC project dedicated only to tracking and versioning datasets and models. The repository would have all the metadata and history of changes in the different datasets.
- This a **data registry**, a middleware between ML projects and cloud storage. This introduces quite a few advantages.
 - Reusability — reproduce and organize feature stores with a simple `dvc get / import`
 - Optimization — track data shared by multiple projects centralized in a single location
 - Data as code — leverage Git workflow such as commits, branching, pull requests, CI/CD ...
 - Persistence — a DVC registry-controlled remote storage improves data security
- But versioning large data files for data science is great, is not all DVC can do: DVC **data pipelines** capture how is data filtered, transformed, and used to train models!

<https://github.com/treeverse/example-get-started-experiments>